

PRÁTICA 05 – APLICAÇÃO COM PERSISTÊNCIA POLIGLOTA

Implementação e Gerenciamento de Bancos de Dados NoSQL

Código no GitHub: github.com/joseniltoncirne/Implementacao-e-Geren-de-banco-de-dados/tree/main/individuais/pratica05-poliglota

O que é

É a mesma página da Prática 04 (OpenF1 Data Explorer), agora usando dois bancos de dados ao mesmo tempo, cada um para o que faz melhor. Isso é a persistência poliglota:

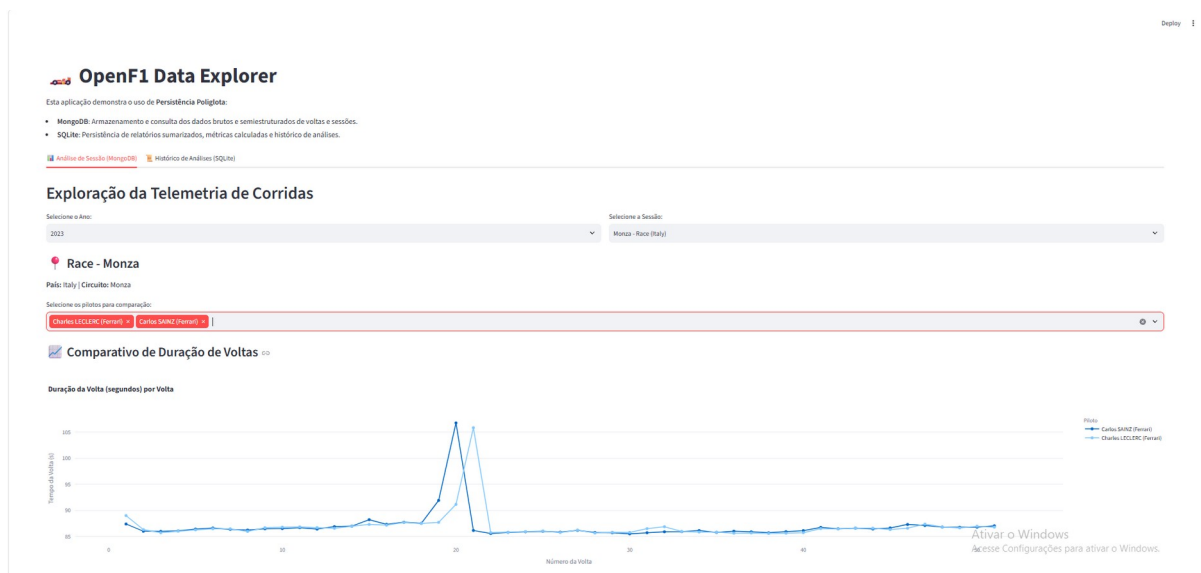
- **MongoDB:** guarda os dados brutos da corrida (sessões, pilotos e cada volta), que são muitos e sem formato fixo.
- **SQLite:** guarda os resumos que a pessoa gera na página (volta mais rápida, tempo médio, total de voltas e regularidade), que são poucos e sempre no mesmo formato, como uma tabela.

Usei o código do professor com alguns ajustes: os pilotos aparecem pelo nome (antes apareciam só os números), o menu mostra só as corridas (antes abria num treino sem voltas) e, no cálculo da regularidade, saem tanto a volta de saída quanto a de entrada dos boxes, como o enunciado pede. Não usei o seed_db.py porque ele apagaria os dados reais da corrida e colocaria dados inventados no lugar.

1. Instalando e abrindo a página

```
python -m pip install -r requirements.txt
python -m streamlit run streamlit_app.py
```

Escolhi o ano 2023, a corrida de Monza (Itália) e os dois pilotos da Ferrari, Charles Leclerc e Carlos Sainz. O gráfico mostra o tempo de cada volta dos dois, buscado no MongoDB. Os picos perto da volta 20 são as paradas nos boxes.



2. Gerando o resumo e salvando no SQLite

Ao clicar em "Gerar Resumo da Análise", a página calcula os números de cada piloto e salva na tabela `race_analysis` do arquivo `analysis_reports.db` (SQLite). O Sainz teve a volta mais rápida (85,50 s) e foi um pouco mais regular (0,61 contra 0,69 do Leclerc).

> Ver Dados Brutos das Voltas

Persistência Poliglota: Gravar Insights no SQLite

Ao clicar no botão abaixo, a aplicação calculará métricas agregadas (MySQL + Relacional) e salvará na tabela SQLite `race_analysis`.

Gerar Resumo da Análise

Métricas Calculadas:

session_name	driver_name	fastest_lap	average_lap_time	total_laps	consistency_pct
0 Monza - Race (Italy)	Charles LECLERC (Ferrari)		85.58	86.917	52
1 Monza - Race (Italy)	Carlos SAINZ (Ferrari)		85.501	86.914	52

☒

 Relatório gravado com sucesso no banco relacional SQLite `[analysis_reports.db]`

3. Histórico de análises

Na aba "Histórico de Análises" a página lê direto do SQLite tudo o que já foi salvo. Esses dados ficam guardados mesmo depois de fechar a página.

OpenF1 Data Explorer

Esta aplicação demonstra o uso de Persistência Poliglota

- MongoDB: Armazenamento e consulta dos dados brutos e semiestruturados de voltas e sessões.
- SQLite: Persistência de relatórios sumariados, métricas calculadas e histórico de análises.

Análise de Sessão (MongoDB)

Histórico de Análises (SQLite)

Histórico de Análises Armazenadas (SQLite)

Exibição direta dos dados agregados e persistidos na tabela `race_analysis` do banco de dados relacional.

ID	Sessão	Piloto	Volta mais rápida (s)	Tempo Médio (s)	Total de Voltas	Consistência (Desvio Padrão)	Data/Hora da Análise
0	2 Monza - Race (Italy)	Carlos SAINZ (Ferrari)		85.501	86.914	52	0.61 2026-09-28 03:12:54
1	1 Monza - Race (Italy)	Charles LECLERC (Ferrari)		85.58	86.917	52	0.687 2026-09-28 03:12:54

Total de Análises Salvas

Sessões Estatísticas Analisadas

Pilotos Processados

2

1

2

Código

streamlit_app.py

```
import streamlit as st
import pandas as pd
import plotly.express as px
from db_utils import (
    fetch_years,
    fetch_sessions_by_year,
    fetch_laps_data,
    fetch_driver_names,
    save_analysis_reports,
    fetch_analysis_history
)

st.set_page_config(
    page_title="OpenF1 Data Explorer",
    page_icon="🏎️",
    layout="wide"
)

st.title("🏎️ OpenF1 Data Explorer")
st.markdown("""
Esta aplicação demonstra o uso de Persistência Poliglota:
- MongoDB: Armazenamento e consulta dos dados brutos e semiestruturados de voltas e sessões.
- SQLite: Persistência de relatórios sumarizados, métricas calculadas e histórico de análises.
""")

tab1, tab2 = st.tabs(["🇧🇷 Análise de Sessão (MongoDB)", "📜 Histórico de Análises (SQLite)"])

with tab1:
    st.header("Exploração da Telemetria de Corridas")

    try:
        available_years = fetch_years()
    except Exception as e:
        st.error(f"Erro ao conectar ao MongoDB. Verifique suas credenciais em `.env`.")
        st.stop()

    if not available_years:
        st.warning("Nenhum dado de ano encontrado na coleção 'sessions'.")
        st.stop()

    col_filter1, col_filter2 = st.columns(2)

    with col_filter1:
        selected_year = st.selectbox("Selecione o Ano:", options=available_years)

        sessions = fetch_sessions_by_year(selected_year)

        if not sessions:
            st.info("Nenhuma sessão encontrada para o ano selecionado.")
            st.stop()

        session_map = {
            f"{s.get('location', 'N/A')} - {s.get('session_name', 'Sessão')}"
            (s.get('country_name', '')): s
            for s in sessions
        }

        with col_filter2:
            selected_session_label = st.selectbox("Selecione a Sessão:",
options=list(session_map.keys()))

            selected_session = session_map[selected_session_label]

            st.subheader(f"📍 {selected_session.get('session_name')} - {selected_session.get('location')}")
            st.write(f"País: {selected_session.get('country_name', 'N/A')} | Circuito: {selected_session.get('circuit_short_name', 'N/A')}")
```

```

df_laps = fetch_laps_data(selected_session["session_key"])

if df_laps.empty:
    st.warning("Não há registros de voltas cadastrados para esta sessão.")
    st.stop()

driver_col = "driver_number" if "driver_number" in df_laps.columns else "driver_name"
lap_duration_col = "lap_duration" if "lap_duration" in df_laps.columns else "duration"
lap_num_col = "lap_number" if "lap_number" in df_laps.columns else "lap"

available_drivers = sorted(df_laps[driver_col].dropna().unique())
driver_names = fetch_driver_names(selected_session["session_key"])
nome_piloto = lambda d: driver_names.get(d, str(d))

selected_drivers = st.multiselect(
    "Selecione os pilotos para comparação:",
    options=available_drivers,
    default=available_drivers[:2] if len(available_drivers) >= 2 else available_drivers,
    format_func=nome_piloto
)

if selected_drivers:
    df_filtered = df_laps[df_laps[driver_col].isin(selected_drivers)].copy()
    df_filtered["piloto"] = df_filtered[driver_col].map(nome_piloto)
    df_filtered = df_filtered.sort_values(["piloto", lap_num_col])

    st.subheader("📈 Comparativo de Duração de Voltas")

    fig = px.line(
        df_filtered,
        x=lap_num_col,
        y=lap_duration_col,
        color="piloto",
        markers=True,
        title="Duração da Volta (segundos) por Volta",
        labels={
            lap_num_col: "Número da Volta",
            lap_duration_col: "Tempo da Volta (s)",
            "piloto": "Piloto"
        }
    )
    fig.update_layout(hovermode="x unified")
    st.plotly_chart(fig, width="stretch")

    with st.expander("📄 Ver Dados Brutos das Voltas"):
        st.dataframe(df_filtered, width="stretch")

    st.divider()

    st.subheader("💾 Persistência Poliglota: Gravar Insights no SQLite")
    st.caption("Ao clicar no botão abaixo, a aplicação calculará métricas agregadas (NoSQL -> Relacional) e salvará na tabela SQLite `race_analysis`.")

    if st.button("🚀 Gerar Resumo da Análise", type="primary"):
        summary_list = []

        for driver in selected_drivers:
            df_driver = df_filtered[df_filtered[driver_col] == driver].copy()

            df_driver_valid = df_driver[df_driver[lap_duration_col] > 0]

            if "is_pit_out_lap" in df_driver_valid.columns:
                voltas_saida = df_driver.loc[df_driver["is_pit_out_lap"] == True,
                lap_num_col]
                voltas_box = set(voltas_saida) | set(voltas_saida - 1)
                df_driver_consistency =
df_driver_valid[~df_driver_valid[lap_num_col].isin(voltas_box)]
            else:
                df_driver_consistency = df_driver_valid

            fastest_lap = df_driver_valid[lap_duration_col].min() if not
df_driver_valid.empty else None
            avg_lap_time = df_driver_valid[lap_duration_col].mean() if not
df_driver_valid.empty else None
            total_laps = len(df_driver)

```

```

        std_dev = df_driver_consistency[lap_duration_col].std() if
len(df_driver_consistency) > 1 else 0.0

        summary_list.append({
            "session_name": selected_session_label,
            "driver_name": nome_piloto(driver),
            "fastest_lap": round(fastest_lap, 3) if fastest_lap else None,
            "average_lap_time": round(avg_lap_time, 3) if avg_lap_time else None,
            "total_laps": total_laps,
            "consistency_std_dev": round(std_dev, 3) if std_dev else 0.0
        })

    st.write("***Métricas Calculadas***")
    df_summary = pd.DataFrame(summary_list)
    st.dataframe(df_summary, width="stretch")

    try:
        save_analysis_reports(summary_list)
        st.success("✅ Relatório gravado com sucesso no banco relacional SQLite
(`analysis_reports.db`)!")
    except Exception as e:
        st.error(f"Erro ao salvar os dados no SQLite: {e}")

    else:
        st.info("Por favor, selecione ao menos um piloto para gerar as métricas e o gráfico.")

with tab2:
    st.header("📜 Histórico de Análises Armazenadas (SQLite)")
    st.markdown("Exibição direta dos dados agregados e persistidos na tabela
**`race_analysis`** do banco de dados relacional.")

    try:
        df_history = fetch_analysis_history()

        if not df_history.empty:
            df_history = df_history.rename(columns={
                "id": "ID",
                "session_name": "Sessão",
                "driver_name": "Piloto",
                "fastest_lap": "Volta mais Rápida (s)",
                "average_lap_time": "Tempo Médio (s)",
                "total_laps": "Total de Voltas",
                "consistency_std_dev": "Consistência (Desvio Padrão)",
                "analysis_timestamp": "Data/Hora da Análise"
            })

            st.dataframe(df_history, width="stretch")

            st.divider()
            col_m1, col_m2, col_m3 = st.columns(3)
            col_m1.metric("Total de Análises Salvas", len(df_history))
            col_m2.metric("Sessões Distintas Analisadas", df_history["Sessão"].nunique())
            col_m3.metric("Pilotos Processados", df_history["Piloto"].nunique())

        else:
            st.info("Nenhum histórico encontrado. Realize uma análise na primeira aba e clique
em 'Gerar Resumo da Análise'.")
    except Exception as e:
        st.error(f"Erro ao carregar o histórico do SQLite: {e}")

```

db_utils.py

```

import os
import pandas as pd
from datetime import datetime
from dotenv import load_dotenv
from pymongo import MongoClient
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime, inspect
from sqlalchemy.orm import declarative_base, sessionmaker

load_dotenv()

MONGODB_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017/")
MONGODB_DB_NAME = os.getenv("MONGODB_DB_NAME", "openf1_data")
SQLITE_DB_PATH = os.getenv("SQLITE_DB_PATH", "sqlite:///analysis_reports.db")

```

```

def get_mongo_client():
    return MongoClient(MONGODB_URI)

def get_mongo_db():
    client = get_mongo_client()
    return client[MONGODB_DB_NAME]

def fetch_years():
    db = get_mongo_db()
    sessions_col = db["sessions"]
    years = sessions_col.distinct("year")
    return sorted(years, reverse=True)

def fetch_sessions_by_year(year: int):
    db = get_mongo_db()
    sessions_col = db["sessions"]
    cursor = sessions_col.find({"year": year, "session_name": "Race"}, {"_id": 0,
"session_key": 1, "location": 1, "country_name": 1, "session_name": 1, "circuit_short_name":
1})
    return list(cursor)

def fetch_driver_names(session_key: int):
    db = get_mongo_db()
    cursor = db["drivers"].find({"session_key": session_key}, {"_id": 0, "driver_number": 1,
"full_name": 1, "team_name": 1})
    return {d["driver_number"]: f"{d.get('full_name', d['driver_number'])}"
({d.get('team_name', '')})" for d in cursor}

def fetch_laps_data(session_key: int):
    db = get_mongo_db()
    laps_col = db["laps"]
    cursor = laps_col.find({"session_key": session_key}, {"_id": 0})
    df = pd.DataFrame(list(cursor))
    return df

Base = declarative_base()

class RaceAnalysis(Base):
    __tablename__ = "race_analysis"

    id = Column(Integer, primary_key=True, autoincrement=True)
    session_name = Column(String(255), nullable=False)
    driver_name = Column(String(255), nullable=False)
    fastest_lap = Column(Float, nullable=True)
    average_lap_time = Column(Float, nullable=True)
    total_laps = Column(Integer, nullable=False)
    consistency_std_dev = Column(Float, nullable=True)
    analysis_timestamp = Column(DateTime, default=datetime.now)

engine = create_engine(SQLITE_DB_PATH, echo=False)
SessionLocal = sessionmaker(bind=engine)

def init_sqlite_db():
    Base.metadata.create_all(engine)

def save_analysis_reports(reports_data: list[dict]):
    init_sqlite_db()
    db_session = SessionLocal()
    try:
        for item in reports_data:
            record = RaceAnalysis(
                session_name=item["session_name"],
                driver_name=str(item["driver_name"]),
                fastest_lap=float(item["fastest_lap"]) if pd.notnull(item["fastest_lap"]) else
None,
                average_lap_time=float(item["average_lap_time"]) if
pd.notnull(item["average_lap_time"]) else None,
                total_laps=int(item["total_laps"]),
                consistency_std_dev=float(item["consistency_std_dev"]) if
pd.notnull(item["consistency_std_dev"]) else None,
                analysis_timestamp=datetime.now()
            )
            db_session.add(record)

```

```

        db_session.commit()
    except Exception as e:
        db_session.rollback()
        raise e
    finally:
        db_session.close()

def fetch_analysis_history():
    init_sqlite_db()
    db_session = SessionLocal()
    try:
        query =
db_session.query(RaceAnalysis).order_by(RaceAnalysis.analysis_timestamp.desc())
        df = pd.read_sql(query.statement, db_session.bind)
        return df
    finally:
        db_session.close()

```

requirements.txt

```

streamlit>=1.30.0
pymongo>=4.6.0
pandas>=2.0.0
sqlalchemy>=2.0.0
plotly>=5.18.0
python-dotenv>=1.0.0

```

.env

```

MONGODB_URI=mongodb://localhost:27017/
MONGODB_DB_NAME=openfl_data
SQLITE_DB_PATH=sqlite:///analysis_reports.db

```